

Белорусский государственный университет  
информатики и радиоэлектроники  
Институт информационных технологий  
Кафедра информационных систем и технологий

**КОНТРОЛЬНАЯ РАБОТА**  
по дисциплине  
**«ТЕХНОЛОГИИ ВЕБ-ПРОГРАММИРОВАНИЯ»**  
Вариант № 25

Выполнил студент группы 381574

Сушко Алексей Юльевич

Подпись \_\_\_\_\_

Проверил:

Ст. преподаватель кафедры ИСиТ

Матвеев Андрей Владимирович

Дата защиты \_\_\_\_\_

Оценка \_\_\_\_\_

Подпись \_\_\_\_\_

Минск, 2025

## Выполнение работы

### Register.cshtml

@page

@model RegisterModel

@{

     ViewData["Title"] = "Register";

}

<h1>@ViewData["Title"]</h1>

<div class="row">

    <div class="col-md-4">

        <form        asp-route-returnUrl="@Model.ReturnUrl"        method="post"  
enctype="multipart/form-data">

        <h4>Create a new account.</h4>

        <hr />

        <div asp-validation-summary="All" class="text-danger"></div>

        <div class="form-group">

            <label asp-for="Input.Email"></label>

            <input asp-for="Input.Email" class="form-control" />

            <span asp-validation-for="Input.Email" class="text-danger"></span>

        </div>

        <div class="form-group">

            <label asp-for="Input.Password"></label>

            <input asp-for="Input.Password" class="form-control" />

            <span        asp-validation-for="Input.Password"        class="text-  
danger"></span>

        </div>

        <div class="form-group">

            <label asp-for="Input.ConfirmPassword"></label>

            <input asp-for="Input.ConfirmPassword" class="form-control" />

            <span        asp-validation-for="Input.ConfirmPassword"        class="text-  
danger"></span>

        </div>

```

        <div class="form-group">
            <label asp-for="Input.Avatar"></label>
            <input asp-for="Input.Avatar" type="file" class="form-control" />
        </div>
        <button type="submit" class="btn btn-primary">Register</button>
    </form>
</div>
</div>

```

@section Scripts {

```

    <partial name="_ValidationScriptsPartial" />
    <script src="~/lib/bs-custom-file-input/dist/bs-custom-fileinput.js"></script>
    <script>
        $(document).ready(function() {
            bsCustomFileInput.init()
        })
    </script>
}

```

### **Register.cshtml.cs**

```

using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.IO;
using System.Text.Encodings.Web;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.UI.Services;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.Extensions.Logging;

```

```
using WebLabsV05.DAL.Entities;
```

```
using System.Security.Claims;
```

```
namespace WebLabsV05.Areas.Identity.Pages.Account
```

```
{
```

```
    [AllowAnonymous]
```

```
    public class RegisterModel : PageModel
```

```
    {
```

```
        private readonly SignInManager<ApplicationUser> _signInManager;
```

```
        private readonly UserManager<ApplicationUser> _userManager;
```

```
        private readonly ILogger<RegisterModel> _logger;
```

```
        private readonly IEmailSender _emailSender;
```

```
        public RegisterModel(
```

```
            UserManager<ApplicationUser> userManager,
```

```
            SignInManager<ApplicationUser> signInManager,
```

```
            ILogger<RegisterModel> logger,
```

```
            IEmailSender emailSender)
```

```
        {
```

```
            _userManager = userManager;
```

```
            _signInManager = signInManager;
```

```
            _logger = logger;
```

```
            _emailSender = emailSender;
```

```
        }
```

```
        [BindProperty]
```

```
        public InputModel Input { get; set; }
```

```
        public string returnUrl { get; set; }
```

```
        public class InputModel
```

```
        {
```

```
            [Required]
```

```

    [EmailAddress]
    [Display(Name = "Email")]
    public string Email { get; set; }

    [Required]
    [StringLength(100, ErrorMessage = "The {0} must be at least {2} and at
max {1} characters long.", MinimumLength = 6)]
    [DataType(DataType.Password)]
    [Display(Name = "Password")]
    public string Password { get; set; }

    [DataType(DataType.Password)]
    [Display(Name = "Confirm password")]
    [Compare("Password", ErrorMessage = "The password and confirmation
password do not match.")]
    public string ConfirmPassword { get; set; }

    public IFormFile Avatar { get; set; }

}

public void OnGet(string returnUrl = null)
{
    ReturnUrl = returnUrl;
}

public async Task<IActionResult> OnPostAsync(string returnUrl = null)
{
    returnUrl = returnUrl ?? Url.Content("~/");
    if (ModelState.IsValid)
    {
        var user = new ApplicationUser { UserName = Input.Email, Email =
Input.Email };
        if(Input.Avatar!=null)

```

```

    {
        user.AvatarImage = new byte[(int)Input.Avatar.Length];
        await
Input.Avatar.OpenReadStream().ReadAsync(user.AvatarImage,0,(int)Input.Avatar.
Length);
        user.ImageMimeType = Input.Avatar.ContentType;
    }

var result = await _userManager.CreateAsync(user, Input.Password);

if (result.Succeeded)
{
    _logger.LogInformation("User created a new account with
password.");

    //var code = await
_userManager.GenerateEmailConfirmationTokenAsync(user);
    //var callbackUrl = Url.Page(
    //    "/Account/ConfirmEmail",
    //    pageHandler: null,
    //    values: new { userId = user.Id, code = code },
    //    protocol: Request.Scheme);

    //await _emailSender.SendEmailAsync(Input.Email, "Confirm your
email",
    //    $"Please confirm your account by <a
href='{HtmlEncoder.Default.Encode(callbackUrl)}'>clicking here</a>.");

    await _signInManager.SignInAsync(user, isPersistent: false);
    return LocalRedirect(returnUrl);
}
foreach (var error in result.Errors)
{
    ModelState.AddModelError(string.Empty, error.Description);
}

```

```

        }
    }

    // If we got this far, something failed, redisplay form
    return Page();
}
}
}

```

### **ImageController.cs**

```

using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Identity;
using WebLabsV05.DAL.Entities;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.StaticFiles;

namespace WebLabsV05.Controllers
{
    public class ImageController : Controller
    {
        UserManager<ApplicationUser> _userManager;
        IHostingEnvironment _env;

        public ImageController(UserManager<ApplicationUser> mngr,
            IHostingEnvironment env)
        {
            _userManager = mngr;
            _env = env;
        }

        public async Task<IActionResult> GetAvatar()
        {

```

```

var user = await _userManager.GetUserAsync(User);
if(user.AvatarImage!=null)
    return File(user.AvatarImage, user.ImageMimeType);
else
{
    var avatarPath = "/Images/anonymous.png";
    var extProvider = new FileExtensionContentTypeProvider();
    var mimeType = extProvider.Mappings[".png"];
    return
File(_env.WebRootFileProvider.GetFileInfo(avatarPath).CreateReadStream(),
        mimeType);
}
}
}
}

```

### **Dish.cs**

```

using System;
using System.Collections.Generic;
using System.Text;

namespace WebLabsV05.DAL.Entities
{
    public class Dish
    {
        public int DishId { get; set; } // id блюда
        public string DishName { get; set; } // название блюда
        public string Description { get; set; } // описание блюда
        public int Calories { get; set; } // кол. калорий на порцию
        public string Image { get; set; } // имя файла изображения

        // Навигационные свойства
        /// <summary>

```



```

        /// группа блюд (например, супы, напитки и т.д.)
        /// </summary>
        public int DishGroupId { get; set; }
        public DishGroup Group { get; set; }
    }
}

```

### **DishGroup.cs**

```

using System;
using System.Collections.Generic;
using System.Text;

namespace WebLabsV05.DAL.Entities
{
    public class DishGroup
    {
        public int DishGroupId { get; set; }
        public string GroupName { get; set; }
        /// <summary>
        /// Навигационное свойство 1-ко-многим
        /// </summary>
        public List<Dish> Dishes { get; set; }
    }
}

```

### **ProductController.cs**

```

using System.Collections.Generic;
using System.Linq;
using Microsoft.AspNetCore.Mvc;
using WebLabsV05.DAL.Entities;
using WebLabsV05.Models;
using WebLabsV05.Extensions;
using WebLabsV05.DAL.Data;

```

```
using Microsoft.Extensions.Logging;
```

```
namespace WebLabsV05.Controllers
```

```
{
```

```
    public class ProductController : Controller
```

```
    {
```

```
        public List<Dish> _dishes { get; set; }
```

```
        List<DishGroup> _dishGroups;
```

```
        ApplicationDbContext _context;
```

```
        private ILogger _logger;
```

```
        int _pageSize;
```

```
        public ProductController(ApplicationDbContext context,
```

```
                                ILogger<ProductController> logger)
```

```
        {
```

```
            _pageSize = 3;
```

```
            _context = context;
```

```
            _logger = logger;
```

```
            //SetupData();
```

```
        }
```

```
        [Route("Catalog")]
```

```
        [Route("Catalog/Page_{pageNo=1}")]
```

```
        public IActionResult Index(int? group, int pageNo)
```

```
        {
```

```
            var groupMame = group.HasValue
```

```
                ? _context.DishGroups.Find(group.Value)?.GroupName
```

```
                : "all groups";
```

```
            //          _logger.LogInformation("info:          group={groupMame},  
page={pageNo}",groupMame,pageNo);
```

```
            var dishesFiltered = _context.Dishes
```

```
                .Where(d => !group.HasValue || d.DishGroupId == group.Value);
```

```

// Поместить список групп во ViewData
ViewData["Groups"] = _context.DishGroups;

// Получить id текущей группы и поместить в TempData
var currentGroup = group.HasValue
    ? group.Value
    : 0;

ViewData["CurrentGroup"] = currentGroup;

//if (Request.Headers["x-requested-with"] == "XMLHttpRequest")
//if (Request.IsAjaxRequest())
//    return PartialView("_ListPartial",
//        ListViewModel<Dish>.GetModel(dishesFiltered, pageNo, _pageSize));
//return View(ListViewModel<Dish>.GetModel(dishesFiltered, pageNo,
//    _pageSize));
}

/// <summary>
/// Инициализация списков
/// </summary>
private void SetupData()
{
    _dishGroups = new List<DishGroup>
    {
        new DishGroup {DishGroupId=1, GroupName="Стартеры"},
        new DishGroup {DishGroupId=2, GroupName="Салаты"},
        new DishGroup {DishGroupId=3, GroupName="Супы"},
        new DishGroup {DishGroupId=4, GroupName="Основные блюда"},
        new DishGroup {DishGroupId=5, GroupName="Напитки"},
        new DishGroup {DishGroupId=6, GroupName="Десерты"}
    };
}

```

```

_dishes = new List<Dish>
{
    new Dish { DishId = 1, DishName="Суп-харчо", Description="Очень
острый, невкусный",
        Calories =200, DishGroupId=3, Image="Суп.jpg" },
    new Dish {DishId = 2, DishName="Борщ", Description="Много
сала, без сметаны",
        Calories =330, DishGroupId=3, Image="Борщ.jpg" },
    new Dish {DishId = 3, DishName="Котлета пожарская",
Description="Хлеб - 80%, Морковь - 20%",
        Calories =635, DishGroupId=4, Image="Котлета.jpg" },
    new Dish {DishId = 4 ,DishName="Макароны по-флотски",
Description="С охотничьей колбаской",
        Calories =524, DishGroupId=4, Image="Макароны.jpg" },
    new Dish {DishId = 5, DishName="Компот",
Description="Быстро растворимый, 2 литра",
        Calories =180, DishGroupId=5, Image="Компот.jpg" }
};
}
}
}
}

```

#### 4 Результат выполнения программы (скриншоты)

WebLabsV05
Lab 3
Каталог
Администрирование
API-demo
0 калорий(0)

### Index

Добавить

|  | DishName            | Description                 | Calories | Group          |  |  |
|--|---------------------|-----------------------------|----------|----------------|--|--|
|  | Суп-харчо           | Очень острый, невкусный     | 200      | Супы           |  |  |
|  | Борщ                | Много сала, без сметаны     | 330      | Супы           |  |  |
|  | Котлета пожарская   | Хлеб - 80%, Морковь - 20%   | 635      | Основные блюда |  |  |
|  | Макароны по-флотски | С охотничьей колбаской      | 524      | Основные блюда |  |  |
|  | Компот              | Быстро растворимый, 2 литра | 180      | Напитки        |  |  |

Сушко КРКаталогАдминистрированиеAPI-demo

1165 калорий(3)🛒👤

Все ▾



Суп-харчо

Очень острый, невкусный

200 калорий

🛒 В корзину



Борщ

Много сала, без сметаны

330 калорий

🛒 В корзину



Котлета пожарская

Хлеб - 80%, Морковь - 20%

635 калорий

🛒 В корзину

12

f vk tw

Сушко КРКаталогАдминистрированиеAPI-demo

1165 калорий(3)🛒👤

Ваша корзина

|                   | Quantity |   |
|-------------------|----------|---|
| Суп-харчо         | 1        | ✖ |
| Борщ              | 1        | ✖ |
| Котлета пожарская | 1        | ✖ |

f vk tw